

Communications between script and parent process, and parent process and child processes:

Script and general dispatcher:

script05.sh first builds the project and launches prog05v4 in the background. They communicate through a pipe. The script creates a temporary named pipe using mkfifo. Any time a new .map or .task file appears in the watch folder, the script writes the file's path to that pipe. prog05v4 reads these file paths from its standard input. Each path is sent as a plain text line. This setup allows the Bash script to stay running while the C++ program continuously receives updates and dispatches new skier processes as files appear.

Skier dispatcher and general dispatcher:

The general dispatcher creates unnamed pipes for communication with each skier dispatcher: When a new .map file path is received, the program forks a new child process (the skier dispatcher). The child reads from its end of the pipe and loads the map into memory. The parent keeps the write end to send "MAP", "TASK", and "QUIT" commands. If the message is MAP, then the path to the map is included and same if it is TASK, the path to the task is included.

Skier dispatcher and output files:

Each skier dispatcher writes its results directly to a file in the output directory. The parent and other processes don't read these files, only the dispatcher writes them.

Extra credit:

I did not include any extra credit in the final version

Difficulties:

- macOS uses Bash 3.2, which doesn't support associative arrays or coproc, so I replaced those features with manual array loops and named pipes (mkfifo).
- Using set -u caused issues when arrays were empty. I fixed this by declaring arrays early or looping directly over globs instead of \${FILES[@]}.

Limitations

- The program doesn't stop if a .map file is deleted after the process starts